

Discrete Mathematics Spring 2026

Coding Assignment 3

Due Date: April 12th, 2026

Academic Honesty Policy:

Assignments are **to be completed individually**. Please do not include any of your code snippets or algorithms in public EdStem posts. You cannot not use solutions from any external source. You are not permitted to publish code or solutions online, nor post the course questions on forums including stack overflow. We run plagiarism detection software. If you have any questions at all about this policy please ask the course staff before submitting your assignment.

Read Carefully:

- Feel free to import any standard Python libraries you need as far as output and input of functions meet the requirement. Use the provided template to implement the functions.
- You **must name your file** “coding3.py”. Any submission that does not follow this naming will not be graded and **will receive a zero**.
- You **must make sure your file extension is .py**. Remember that online environments like Google Colab might export a .ipynb file; this is **not the correct filetype**. Colab (and other environments) have the option to export as .py. If you are using some sort of Python notebook, *make sure your code compiles in a single block before exporting*.
- A skeleton of each function has been provided to you. **You are expected to ONLY write code in the functions and blocks specified**. In any case, DO NOT modify the function signatures, return variables, or any code that is not specified to be modifiable (we will include comments in the code to distinguish these sections) for any reason. Also, though you may modify the `main` function to test other cases, **do not turn in an assignment with a modified main function, i.e. anything below the line shown below**. This will be run by our autograder, so any unexpected modifications that make it malfunction **will receive a zero**.

```
### DO NOT TURN IN AN ASSIGNMENT WITH ANYTHING BELOW HERE MODIFIED ###  
if __name__ == "__main__":
```

- Test cases will be provided in the `main` function in the code. Several of the test cases for each part will be provided to you, and the others will be hidden test cases on our side (all graded via autograder).
- **Only Python 3.x versions will be graded**. To check your Python version locally, open a terminal window and enter:

```
python --version
```

Google Colab should use Python 3 by default, but, to check, under “Runtime,” navigate to “Change runtime type.”

- To receive points, make sure your code runs. We recommend using Spyder, Pycharm or Google colab. They all allow you to download .py files. Be aware that if you write your code in some platforms like Codio and copy and paste it in a text file, there may be spurious characters in your code, and your code will not compile. **Always ensure that your .py compiles. Code that does not run will not receive any points.**

Part A: Relation Properties

In the following section, we will assess the properties of relations depicted as matrices. The function will receive only one input, the adjacency matrix of the graph representing the relation, for example:

- ```
adj_mat = [[1,0,1],
 [0,0,1],
 [1,1,1]]
```

Here the relation can be represented by the pairs  $\{(1,1), (1,3), (2,3), (3,1), (3,2), (3,3)\}$  since an element of the adjacency matrix in the  $i^{\text{th}}$  row and  $j^{\text{th}}$  column is only 1 if the pair  $(i,j)$  is in the relation

- (a) Write a function to check whether an inputted relation is reflexive. The python function should be called `is_reflexive`, should take in the adjacency matrix argument representing the relation, and should return a boolean, `True` or `False`.

```
def is_reflexive(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (b) Write a function to check whether an inputted relation is symmetric. The python function should be called `is_symmetric`, should take in the adjacency matrix arguments representing the relation, and should return a boolean, `True` or `False`.

```
def is_symmetric(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (c) Write a function to check whether an inputted relation is transitive. The python function should be called `is_transitive`, should take in the adjacency matrix arguments representing the relation, and should return a boolean, `True` or `False`.

```
def is_transitive(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (d) Write a function to check whether an inputted relation is irreflexive. The python function should be called `is_irreflexive`, should take in the adjacency matrix arguments representing the relation, and should return a boolean, `True` or `False`.

```
def is_irreflexive(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (e) Write a function to check whether an inputted relation is antisymmetric. The python function should be called `is_antisymmetric`, should take in the adjacency matrix arguments representing the relation, and should return a boolean, `True` or `False`.

```
def is_antisymmetric(adj_mat):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

Note: We did not provide any test cases for antisymmetric or irreflexive, as we would like you to try and test it yourself. Writing test cases is an important part of coding and it is good practice not learn how to test your code. Please feel free to test your code with whatever examples you could come up with, but when you submit DO NOT ALTER THE MAIN METHOD

## Part B: Function Properties

In the following sections we will assess the properties of an inputted function. The function will be inputted in 3 parts:

- A list of the values in the domain For example:

```
domain = [1,2,3,4]
```

- A list of the values in the co-domain

```
co_domain = [1,2,3,4,5,6,7]
```

- A dictionary of the function mapping between the domain and the co-domain For example:

```
mapping = {
 1: 2,
 2: 3,
 3: 1,
 4: 3
}
```

The example domain, co-domain, and mapping given above would represent this function:

- (a) Write a function to check whether an inputted function is onto. The python function should be called `is_onto`, should take in the 3 arguments representing the function (domain, co-domain, and mapping), and should return a boolean, `True` or `False`.

```
def is_onto(domain,co_domain,mapping):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (b) Write a function to check whether an inputted function is one-to-one. The python function should be called `is_one_to_one`, should take in the 3 arguments representing the function (domain, co-domain, and mapping), and should return a boolean, `True` or `False`.

```
def is_one_to_one(domain,co_domain,mapping):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```

- (c) Write a function to check whether an inputted function is bijective. (Hint: you can utilize your other functions in writing this one!) The python function should be called `is_bijective`, should take in the 3 arguments representing the function (domain, co-domain, and mapping) and should return a boolean, `True` or `False`.

```
def is_bijective(domain,co_domain,mapping):
 # WRITE YOUR CODE HERE
 return True or False (bool).
```